

NAME :D.G.RAGHAVI

COURSE CODE : CSA1013

REG NO: 192521173

COURSE NAME : SOFTWARE

ENGINEERING

CO4-ASSESSMENT TOOL - 02

Title: AI-Based Smart Construction Project Monitoring and Risk Assessment Platform

1.Project Overview

The proposed system is an AI-powered construction monitoring platform that collects data from drones, CCTV cameras, IoT sensors, BIM models, and project-management systems. It uses AI/ML to monitor construction progress, detect safety violations, predict delays, assess risks, and optimize resources. Because the system contains multiple components such as web applications, APIs, AI models, databases, and IoT services, a CI/CD pipeline is required to automate software integration, testing, security verification, and deployment.

2. CI/CD Requirements Analysis

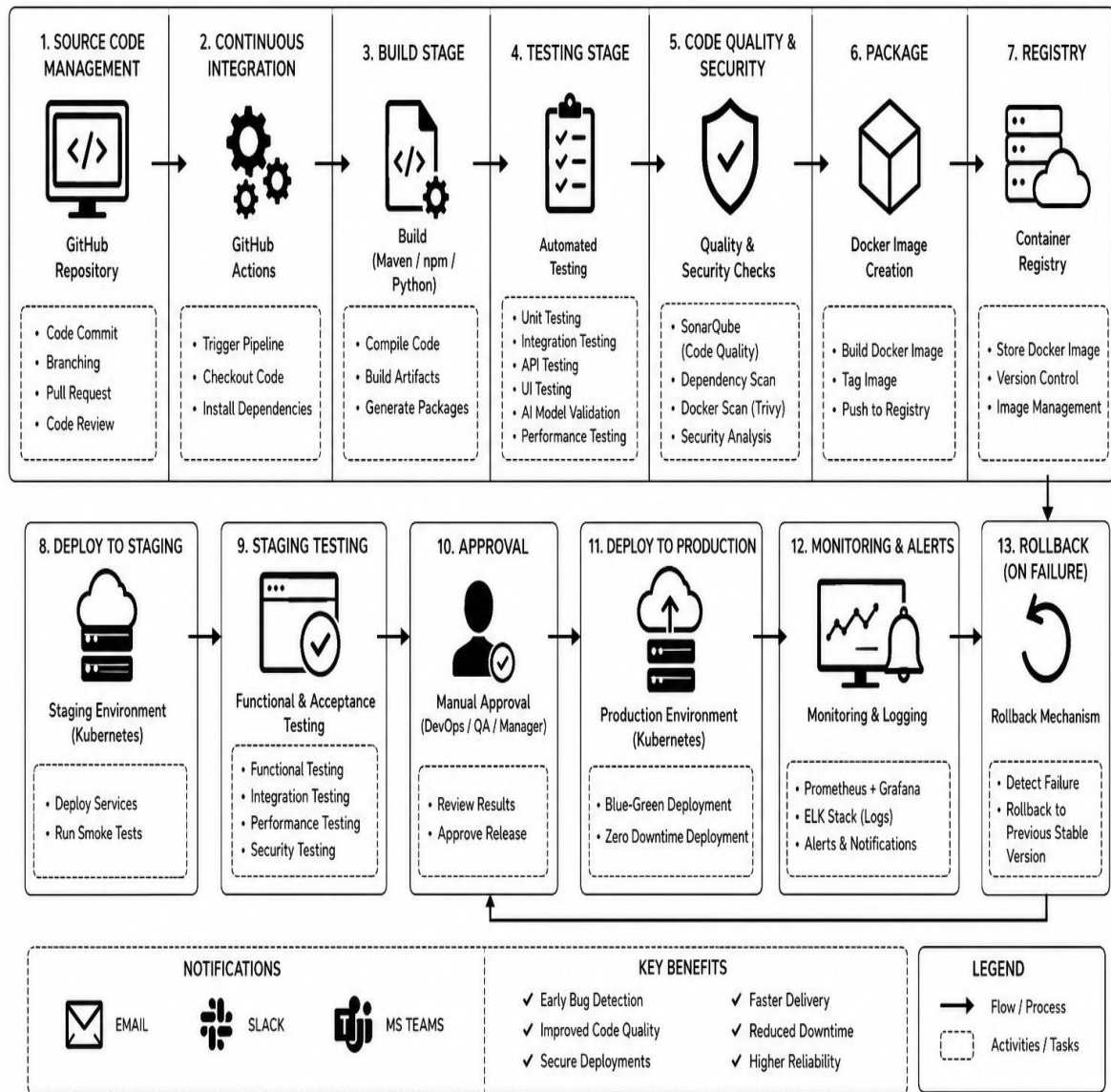
The major CI/CD requirements are:

Requirement	CI/CD Need
Continuous code development	Git-based source control
Multiple developers	Automated code integration
AI/ML models	Model validation and testing
Web/API application	Automated build and deployment
IoT and sensor data	API and integration testing
Safety detection	AI model accuracy testing
Project dashboard	UI testing
Security	Vulnerability and dependency scanning
Reliable deployment	Staging and production environments
High availability	Containerized deployment
Fault recovery	Automated rollback
Continuous monitoring	Logs, alerts and performance monitoring

3. Proposed CI/CD Pipeline Architecture

Pipeline Diagram

CI/CD PIPELINE FOR AI-BASED SMART CONSTRUCTION PROJECT MONITORING AND RISK ASSESSMENT PLATFORM



4. Pipeline Stages and Tools

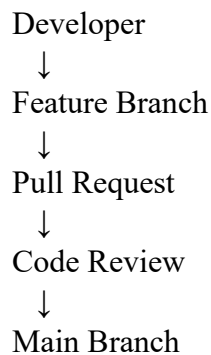
Stage	Tool/Technology	Purpose
Source Code Management	GitHub	Store and manage source code
CI Pipeline	GitHub Actions	Automate pipeline execution
Application Build	Maven / npm / Python	Build application components
Unit Testing	PyTest / JUnit	Test individual components
API Testing	Postman / Newman	Test backend APIs
UI Testing	Selenium	Test dashboard functionality
Code Quality	SonarQube	Detect bugs and code smells

Security Scan	Trivy / OWASP	Detect vulnerabilities
AI Model Testing	PyTest + ML metrics	Validate AI model performance
Containerization	Docker	Package applications
Image Registry	Docker Hub / AWS ECR	Store Docker images
Deployment	Kubernetes	Deploy and manage containers
Infrastructure	AWS / Azure	Cloud infrastructure
Monitoring	Prometheus + Grafana	Monitor system performance
Logs	ELK Stack	Centralized logging
Notifications	Email / Slack	Pipeline status alerts

5. Detailed CI/CD Workflow

Stage 1 – Source Code Management

Developers work on separate Git branches.



GitHub is selected because it provides:

- Version control
- Branch management
- Pull requests
- Code reviews
- GitHub Actions integration

Stage 2 – Continuous Integration

Whenever code is pushed or a pull request is created, GitHub Actions automatically starts the pipeline.

The pipeline:

1. Downloads the latest code.
2. Installs dependencies.
3. Builds the application.
4. Runs automated tests.
5. Performs security and quality checks.

If a critical check fails, the pipeline stops and the developer is notified.

6. Build Stage

The application may contain:

- Frontend
- Backend APIs
- AI/ML services
- Database components
- IoT services

Each component is built automatically.

Example:

Source Code



Install Dependencies



Compile / Package



Build Successful?



Yes



Continue



No



Stop

7. Automated Testing Strategy

Testing is particularly important because incorrect AI predictions or monitoring failures could affect construction safety and project decisions.

A. Unit Testing

Tests individual functions and modules.

Examples:

- User authentication
- Risk calculation
- Progress calculation
- Resource allocation
- Sensor-data processing

Tool: PyTest / JUnit

B. Integration Testing

Checks whether different system components work together.

Examples:

IoT Sensor → API → Database

Drone → Image Processing → AI Model

AI Model → Risk Engine → Dashboard

C. API Testing

Tests REST APIs used by:

- Mobile applications
- Web dashboards
- IoT devices
- AI services

Tool: Postman/Newman

D. UI Testing

The construction manager dashboard is automatically tested.

Tests include:

- Login
- Project creation
- Progress display
- Risk dashboard
- Alert display
- Report generation

Tool: Selenium

E. AI/ML Model Testing

The AI model should be tested using construction-site datasets.

Important metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- False-positive rate
- False-negative rate

For safety-violation detection, recall is particularly important, because missing an actual safety violation can be more serious than generating an additional warning.

F. Performance Testing

Tests whether the platform can handle large amounts of:

- Drone images
- Video streams
- Sensor readings
- User requests
- Construction projects

8. Code Quality and Security

Before deployment, automated quality gates are applied.

Code Quality

SonarQube checks:

- Bugs
- Code smells
- Duplicated code

- Maintainability
- Security issues

Dependency Security

Tools such as OWASP Dependency-Check identify vulnerable libraries.

Container Security

Trivy scans Docker images for vulnerabilities.

Pipeline:

Code



SonarQube



Dependency Scan



Docker Build



Trivy Scan



Security Passed?

If critical vulnerabilities are found, deployment is blocked.

10. Deployment Environments

The system uses three major environments.

Development Environment

Used by developers for coding and initial testing.

Developer → GitHub → Development

Staging Environment

A production-like environment used for final testing.

CI Pipeline



Staging



Integration + Security + Performance Testing

Production Environment

Used by actual construction companies and project managers.

Staging



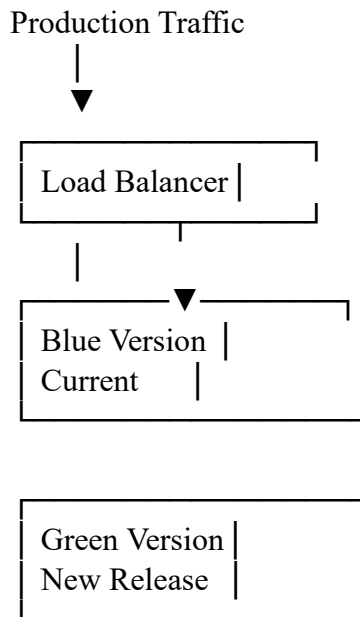
Approval



Production

11. Deployment Strategy

A Blue-Green Deployment strategy can be used.



The new version is deployed to the Green environment and tested.

If successful:

Blue → Green

If problems occur:

Green → Blue

This provides quick rollback with minimal downtime.

12. Security Measures

Security should be incorporated throughout the pipeline.

Security practices

1. Use HTTPS/TLS.
2. Store passwords as encrypted secrets.
3. Never store API keys in source code.
4. Use GitHub Secrets.
5. Scan dependencies.
6. Scan Docker images.
7. Apply role-based access control.
8. Use authentication and authorization.
9. Encrypt sensitive project data.
10. Regularly update dependencies.

13. Monitoring and Notifications

After deployment, the system is continuously monitored.

Prometheus

Collects metrics such as:

- CPU usage
- Memory usage

- API response time
- Application errors
- Container health

Grafana

Displays monitoring dashboards.

ELK Stack

Used for centralized application logs.

Notifications

Pipeline results can be sent to:

- Email
- Slack
- Microsoft Teams

Example:

Build Successful → Notification

Test Failed → Developer Alert

Deployment Failed → DevOps Alert

Production Error → Operations Alert

14. Rollback Mechanism

If a new deployment causes problems, the system automatically or manually returns to the previous stable version.

New Version

↓

Production Monitoring

↓

Error Detected?

/ \

No

Yes

↓

↓

Continue Rollback

↓

Previous Version

Rollback can be triggered by:

- High error rate
- Application crash
- Failed health checks
- Performance degradation
- Critical security issue

15. Example GitHub Actions Workflow

A simplified pipeline configuration can follow this structure:

on:

push:

branches: [main]

Pipeline:

1. Checkout Code
2. Install Dependencies
3. Build Application
4. Run Unit Tests
5. Run SonarQube Analysis
6. Run Security Scan
7. Validate AI Model
8. Run Integration Tests
9. Build Docker Image
10. Scan Docker Image
11. Push Image to Registry
12. Deploy to Staging
13. Run Acceptance Tests
14. Approve Production Deployment
15. Deploy to Production
16. Monitor Application

16. Overall Pipeline

Phase	Activities	Output
Source	Git commit, pull request	Source code
Build	Compile/package application	Build artifact
Test	Unit, API, integration tests	Test report
Quality	SonarQube analysis	Quality report
Security	Dependency/image scanning	Security report
AI Validation	Model performance testing	Validated model
Package	Docker image creation	Container image
Registry	Store image	Versioned image
Staging	Deploy and test	Release candidate
Production	Deploy approved version	Live system
Monitor	Metrics and logs	System health
Rollback	Restore previous version	Stable release

17. Justification of the Proposed Pipeline

The proposed CI/CD pipeline is suitable for the construction monitoring platform because it supports the project's complex combination of AI, IoT, cloud services, APIs, dashboards, and data processing.

GitHub and GitHub Actions provide automated source-code integration and CI/CD. Docker ensures consistent application environments, while Kubernetes supports scalable deployment. Automated unit, integration, API, UI, and AI-model tests improve reliability. SonarQube and security scanners improve code quality and security. Staging deployment allows the new version to be validated before production. Blue-Green deployment provides fast rollback when failures occur.

Therefore, the pipeline reduces manual deployment effort, improves software quality, increases deployment reliability, and supports continuous delivery of new features and AI improvements.

18. Expected Benefits

The CI/CD implementation provides:

1. Faster development through automated builds.
2. Early bug detection through continuous testing.
3. Improved security through automated scanning.
4. Reliable AI model updates through model validation.
5. Reduced deployment errors through automation.
6. Minimal downtime through Blue-Green deployment.
7. Quick recovery through rollback.
8. Continuous monitoring of production services.
9. Better software quality through quality gates.
- 10.** Higher project management efficiency through reliable platform releases.

Conclusion

The proposed CI/CD pipeline provides an automated and secure software delivery process for the AI-Based Smart Construction Project Monitoring and Risk Assessment Platform. It integrates source control, automated building, testing, AI validation, security analysis, containerization, staging, production deployment, monitoring, and rollback.

This approach ensures that updates to the construction monitoring platform can be delivered quickly, safely, consistently, and with minimum downtime, supporting the project's objectives of improved safety, productivity, resource utilization, and decision-making.